

Cookies and Sessions Lab (CodeGrade)

 [_ \(https://github.com/learn-co-curriculum/python-p4-cookies-and-sessions-lab\)](https://github.com/learn-co-curriculum/python-p4-cookies-and-sessions-lab)  [_ \(https://github.com/learn-co-curriculum/python-p4-cookies-and-sessions-lab/issues/new\)](https://github.com/learn-co-curriculum/python-p4-cookies-and-sessions-lab/issues/new)

Learning Goals

- Use the session object to persist data across multiple requests.

Key Vocab

- **Identity and Access Management (IAM)**: a subfield of software engineering that focuses on users, their attributes, their login information, and the resources that they are allowed to access.
- **Authentication**: proving one's identity to an application in order to access protected information; logging in.
- **Authorization**: allowing or disallowing access to resources based on a user's attributes.
- **Session**: the time between a user logging in and logging out of a web application.
- **Cookie**: data from a web application that is stored by the browser. The application can retrieve this data during subsequent sessions.

Introduction

In this lab, you'll be building out a blog paywall feature by using the session hash to keep track of how many page views a user has made.

There is some starter code in place for a Flask API backend and a React frontend. To get set up, run:

```
$ pipenv install && pipenv shell  
$ npm install --prefix client
```

```
$ cd server  
$ flask db upgrade  
$ python seed.py
```

You can work on this lab by running the tests with `pytest -x`. It will also be helpful to see what's happening during the request/response cycle by running the app in the browser. You can run the Flask server with:

```
$ python app.py
```

And you can run React from the root directory in another terminal with:

```
$ npm start --prefix client
```

You don't have to make any changes to the React code to get this lab working.

Instructions

Our app will keep track of how many blog posts a user has viewed by using the `session` object. Each user can view a **maximum of three articles** before seeing the payroll.

When a user makes a `GET` request to `/articles/<int:id>`, the following should happen:

- If this is the first request this user has made, set `session['page_views']` to an initial value of 0.
 - **Hint:** consider using a ternary operator to set this initial value!
- For every request to `/articles/<int:id>`, increment the value of `session['page_views']` by 1.
- If the user has viewed 3 or fewer pages, render a JSON response with the article data.
- If the user has viewed more than 3 pages, render a JSON response including an error message `{'message': 'Maximum pageview limit reached'}`, and a status code of 401 unauthorized.
- An API endpoint at `/clear` is available to clear your session as needed.

Resources

- [API - Flask: class flask.session](https://flask.palletsprojects.com/en/2.2.x/api/#flask.session)  [\(https://flask.palletsprojects.com/en/2.2.x/api/#flask.session\)](https://flask.palletsprojects.com/en/2.2.x/api/#flask.session)

This tool needs to be loaded in a new browser window

Load Cookies and Sessions Lab (CodeGrade) in a new window